

Environnement de Lab :

AWS EC2 et Docker

Lab #0

Dans le but de créer des environnements sécurisés, banalisés et compatibles avec le travail que vous devrez faire dans le cadre de certains Lab du cours de Forensique, vous aurez à utiliser différentes technologies dont :

- Des ressources Cloud dans l'environnement AWS EC2,
- Des conteneurs Docker.

L'objectif de ce Lab est de vous permettre d'être à l'aise sur ces deux points.

Partie 1 : AWS EC2

Objectifs : Être capable

- De créer et d'accéder à des instances AWS EC2 et d'y installer des paquetages,
- Comprendre le processus de gestion des clés ssh pour accéder à un système distant,
- De comprendre le fonctionnement de base d'un groupe de sécurité sur AWS EC2

Étape 1 : Lab AWS

Vous êtes normalement inscrit sur un cours d'AWS Academy appelé : Learner Lab

Connectez-vous à ce cours et effectuez les actions suivantes:

- Allez dans la partie « Modules » et démarrez le Lab
- Lancez une nouvelle instance avec les paramètres suivants :
 - o Nom de l'instance : **UQAC Lab 1**
 - o Type : **Linux 64-bit (x86)**
 - o Distribution : **Ubuntu Server 24.04 LTS (HVM), SSD Volume Type**
 - o Type d'instance : **t3.large**
 - o Clé ssh : **vockey**
- Groupe de sécurité :
 - o Créez un nouveau groupe de sécurité : **UQAC Forensics**
 - o Règles entrantes : ssh de n'importe où
 - o **Note : Les règles sortantes ne devront pas être modifiées dans ce Lab !**
- Stockage : **30 Go**

Étape 2 : Istration distante d'une instance AWS

Afin d'administrer une instance Linux dans le Cloud, on utilise couramment le protocole ssh. Ce protocole d'accès distant permet d'accéder à une instance en ligne de commandes de manière sécurisée et chiffrée.

Cet accès dans le cas d'AWS EC2, se fait, non pas par mot de passe, mais en utilisant un système de clés asymétrique : le poste d'administration (votre machine) dispose de la clé privée et le serveur à administrer, dans le fichier de configuration ssh de l'utilisateur concerné, ajoute la clé publique correspondante.

Dans le contexte d'AWS Academy (et seulement dans ce contexte), votre clé privée est sur votre environnement de Lab AWS. Elle est disponible à partir de l'interface Lab du cours à partir de l'onglet **AWS Details**.

Travail à réaliser :

- Téléchargez sur votre poste de travail la clé privée au format PEM à partir de votre console AWS Academy. La clé publique associée à cette clé privée a automatiquement été incluse lors du déploiement de l'instance
- Sous Windows, ouvrez un *cmd.exe* ou un *Powershell* et connectez-vous à l'instance :
ssh -i clé_privée ubuntu@IP_PUBLIQUE_DE_L'INSTANCE
- Une fois connecté à l'instance :
 - o Mettez à votre instance :
 - ***sudo apt update***
 - ***sudo apt upgrade -y***
 - ***sudo reboot***
 - o La dernière commande redémarre votre instance et vous déconnecte de celle-ci.
 - o Une fois l'instance redémarrée, reconnectez-vous en ssh

Vous allez maintenant installer un service Web sur votre instance et essayer de vous y connecter. La possibilité de vous connecter à une instance au travers d'un tel service sera exploitée lors du Lab #5.

- o Installez un service Apache : ***sudo apt install apache2***
- o Vérifiez l'état du service Apache : ***sudo systemctl status apache2***
- o À partir de votre poste local, essayez de vous connecter à votre service Web
 - Est-ce que cela fonctionne ?
 - Reprenez ce que vous avez mis en place au niveau du groupe de sécurité et modifiez ce dernier afin de rendre la connexion possible
- o Stoppez le service Web en tapant : ***sudo systemctl stop apache2*** et vérifiez qu'il est bien arrêté
- o Désactivez le service Web au démarrage de l'instance : ***sudo systemctl disable apache2***

Étape 3 : Comprendre la politique des instances AWS EC2 vis-à-vis des adresses IP

Dans votre console AWS, sélectionnez l'instance que vous avez déployée

- Relevez l'IP publique de cette instance et son IP privée
- Dans le menu « *État de l'instance* », arrêtez l'instance
- Quel est l'impact sur les IP précédentes ?
- Redémarrez l'instance. Même question.
- Conclusion ?

Partie 2 : Création d'une instance Debian et installation de Docker Engine

Dans cette partie, vous allez paramétrer un service Docker sur votre instance et lancer votre premier conteneur.

Étape 1 : Installation du service Docker

Travail à réaliser :

- Connectez-vous à votre instance en ssh
- Installez le paquetage tmux : ***sudo apt install tmux***
- Installez le paquetage Docker : ***sudo apt install docker.io***
Note: l'installation de Docker au travers de ce paquetage n'est pas considérée comme la meilleure pratique. Il est normalement conseillé de suivre la procédure sur <https://docs.docker.com/>. Pour des raisons de simplicité pour ce Lab, vous utiliserez cependant le paquetage docker.io de la distribution.
- Installez le paquetage Docker Compose : ***sudo apt install docker-compose***
- Vérifiez que le service Docker est bien actif : ***sudo systemctl status docker***
- Vérifiez si des images de conteneur sont disponibles par défaut : ***sudo docker images***
 - o Normalement il ne devrait pas y avoir d'images installées par défaut
 - o Que se passe-t-il si vous essayez la commande précédente sans ***sudo*** ?

Étape 2 : paramétrage de base de Docker et tests

L'exécution de la commande précédente sans ***sudo*** a dû aboutir à un message d'erreur. Dans le principe de délégation de droits, il est généralement intéressant d'éviter que seul root puisse interagir avec le service Docker et donc de déléguer ces droits à un autre utilisateur. Cela est fait en ajoutant le ou les utilisateurs en question au groupe *docker*

- Éditez le fichier */etc/group* et ajoutez simplement ubuntu à la fin de la ligne correspondant au groupe docker
Note : l'éditeur *vi* est installé par défaut sur la plupart des distributions Linux. Si vous préférez un éditeur plus « convivial » (ceci étant bien entendu tout relatif) tel que *nano*, vous pouvez l'installer. Le nom du paquetage est tout simplement *nano*
- Déconnectez-vous de l'instance et reconnectez-vous afin que votre nouveau rôle soit opérationnel
- Testez à nous la commande : ***docker images***
Normalement, vous devriez maintenant être capable d'exécuter la plupart des commandes docker sans prendre l'identité de root et sans utiliser la commande *sudo*.

Étape 3 : création d'un premier conteneur en mode interactif et actions de base

Maintenant que votre environnement Docker est opérationnel, vous allez pouvoir démarrer votre premier conteneur.

Dans une fenêtre *tmux*, téléchargez et exécutez un conteneur Alpine Linux en mode interactif en tapant :

docker run -it --rm alpine /bin/ash

Cette commande téléchargera l'image de la dernière version du conteneur Alpine, démarrera un conteneur basé sur cette image en mode interactif et exécutera le programme */bin/ash*. Vous devriez maintenant avoir un prompt du type : *#*

Manipulez la ligne de commande au sein de votre conteneur afin de comprendre qui vous êtes, ce qui tourne dans votre conteneur et l'organisation du système de fichiers associé. Par exemple, vous pouvez utiliser les commandes suivantes (liste non exhaustive) :

- *id*
- *cd*
- *ls*
- *ps*

Dans une autre fenêtre *tmux*, obtenez la liste des conteneurs actifs : ***docker ps***
Interprétez le résultat. Quel est le nom associé au conteneur?

Dans la fenêtre associée au conteneur, tapez : ***exit***
Quel est le résultat ? Confirmez votre hypothèse en utilisant ***docker ps***

Exécutez à nouveau votre conteneur. Constatez-vous une différence ? Confirmez votre hypothèse en tapant : ***docker images***
Conclusion ?

Quittez votre conteneur et supprimez votre image alpine en tapant : ***docker image rm alpine***
Essayez à nouveau d'exécuter le conteneur. Conclusion ?

Quels sont, à votre avis, les inconvénients et avantages majeurs de lancer votre conteneur avec les options *-it* et *--rm*. Si vous manquez d'idées à ce niveau, essayez d'exécuter votre conteneur, de créer de fichiers, de le quitter et de le réexécuter.

Essayez d'exécuter plusieurs fois votre conteneur Alpine. Que constatez-vous au niveau des systèmes de fichiers et des processus exécutés au sein du conteneur. Quel pourrait être l'impact forensique?

Faites un **ps aux** au sein de chaque conteneur Alpine et sur votre instance AWS. Conclusion? Quel pourrait être l'intérêt en termes de sécurité?

Supprimez vos conteneurs actifs avant de passer à l'étape suivante

Étape 4 : conteneurs et applications réseaux

Vous allez maintenant déployer un service Web *nginx* dans un conteneur sur votre instance Debian.

Exécutez la commande suivante : ***docker run -d -p 80:80 --name nginx nginx***

Testez que vous avez maintenant un service Web accessible sur votre instance AWS.

Essayez de comprendre le rôle des différentes options :

- L'option ***-d***
- L'option ***-p 80:80***
- L'option ***--name nginx***

Nous allons maintenant voir comment gérer l'état d'un conteneur un conteneur :

- Récupérer l'identité du conteneur *nginx* : ***docker ps***
 - Arrêtez ou « killez » le conteneur en utilisant la commande : ***docker stop Container_ID*** ou ***docker kill Container_ID***
Quelle est la différence entre arrêter et « killer » un conteneur ? À votre avis qu'est-ce qui est préférable d'utiliser ?
Remarque : en lieu et place du *Container_ID*, vous pouvez également utiliser le nom du conteneur.
 - Vérifiez que votre service Web n'est plus accessible. Que donne la commande ***docker ps***
 - Essayez d'exécuter à nouveau votre conteneur comme vous l'avez fait précédemment (avec la même ligne de commande). Est-ce que cela fonctionne ?
 - Utilisez la commande : ***docker ps -a***
 - Déduisez-en quel est le problème ?
 - À ce stade, vous avez potentiellement deux options :
 - o Redémarrer le conteneur précédent en utilisant la commande ***docker start***
 - o Détruire le conteneur en utilisant la commande ***docker rm*** et le relancer
- Soyez certains de tester les deux variantes

Étape 5 : Conteneurs et persistance

Nous avons vu dans les étapes précédentes que, par défaut, un conteneur est dit "Stateless", c'est-à-dire qu'il n'a pas de persistance.

Dans certains cas, nous aimerions avoir la possibilité de conserver les données générées au sein d'un conteneur, partager du contenu à partir du système de fichiers de la machine hôte ou même de partager des fichiers entre différents conteneurs.

Nous allons maintenant proposer une solution pour créer de la persistance :

- Prenez soin d'arrêter et de supprimer tout conteneur sur votre instance AWS
- Sur votre instance, créez un répertoire nommé : *mysite*
- Au sein du répertoire *mysite*, créez un fichier *index.html* et mettez-y du contenu HTML afin de créer une page Web
Conseil : faites quelque chose de très simple, ce n'est pas un cours de programmation Web
- Exécutez un nouveau conteneur en tapant :

docker run -d -p 80:80 --name nginx -v /home/ubuntu/mysite:/usr/share/nginx/html:ro nginx

- Conclusion ? N'hésitez à explorer la signification des différentes options utilisées
- Laisser votre conteneur **nginx** tourner pour la partie suivante

Étape 6 : Accès à un conteneur actif

Nous allons maintenant explorer différentes façons d'interagir avec des conteneurs en cours d'exécution.

Lors de votre première exécution du conteneur Alpine, nous avons forcé le mode interactif. Nous allons maintenant exécuter ce même conteneur différemment :

- Vérifiez qu'aucun conteneur Alpine n'est actif
- Exécutez la commande : ***docker run -dit alpine /bin/sh***
Que fait l'option ***-d*** ? Expliquez le résultat obtenu.
Confortez votre compréhension en utilisant ***docker ps***
- Afin de pouvoir interagir avec votre conteneur, une première option est d'attacher votre périphérique d'entrée (votre clavier) au conteneur. Pour se faire tapez :

docker attach Container_ID

Remarque : pour que cette commande fonctionne, il faut bien évidemment disposer d'un conteneur « interactif ».

Une fois votre périphérique d'entrée attaché au conteneur,

- o Si vous tapez ***exit***, le conteneur s'arrêtera. Vous pouvez vérifier ce point en utilisant la commande ***docker ps -a***
- o Si vous désirez uniquement « détacher » votre clavier du conteneur, vous pouvez utiliser la séquence d'échappement : ***ctrl-p, ctrl-q***

Prenez le temps de vérifier ces deux comportements.

- Une seconde option pour interagir avec un conteneur en cours d'exécution est d'utiliser la commande ***exec*** de docker. Tapez : ***docker exec -it Container_ID /bin/ash***
 - o Une fois « connecté » au conteneur, utilisez la commande ***ps*** et interprétez le résultat obtenu
 - o Déconnectez-vous du conteneur en tapant : ***exit***
 - o Est-ce que le conteneur est stoppé ? Expliquez ce point en vous aidant de l'interprétation que vous venez de faire.
- Quel pourrait être l'avantage ou l'intérêt d'utiliser ***exec*** plutôt que d'attacher le périphérique d'entrée au conteneur ? Quelle est la limitation quant à l'utilisation d'***exec*** ?

Vous allez appliquer ce que vous venez d'apprendre pour interagir avec votre conteneur **nginx** :

- Connectez-vous à votre conteneur **nginx** en utilisant ***docker exec***
- Allez dans le répertoire ***/usr/share/nginx/html***
- Quel fichier trouvez-vous ? Êtes-vous capable de le modifier ?

Avant de passer à la partie suivante, supprimez tous les conteneurs encore actifs.

Partie 3 : Utilisation de Docker compose

Docker compose est un outil classiquement utilisé quand on désire mettre en place des services basés sur des conteneurs travaillant ensemble. Son intérêt est de pouvoir définir des règles de déploiement et

de fonctionnement de différents conteneurs sur la base d'un fichier de configuration unique au format yaml (<https://yaml.org/>).

Pour cette partie, vous allez apprendre à :

- Restaurer une image Docker à partir d'une archive
- Déployer un conteneur et un réseau virtuel à l'aide de docker compose et d'un fichier de configuration yaml.

Étape 1 : Restauration d'une image Docker à partir d'une archive

Dans votre instance AWS,

- Récupérez l'archive dont l'URL est fournie sur Moodle sous le nom « Image Alpine Custom » en utilisant la commande : ***wget -O alpine.sshd.tar "URL_donnée_sur_Moodle"***
Important : N'oubliez pas de mettre les guillemets !
- En utilisant la commande ***shasum***, vérifiez le checksum sha-256 de l'archive. Vous devriez obtenir :
`5f3091a8b8d377eefa87911620c688915f255bd9773a8d3c2301d471d8206887`
- Restaurez l'image Docker en tapant : ***docker load < alpine.sshd.tar***
- Vérifiez que l'image est bien présente sur votre instance. Quel est le nom de l'image importée ?

Étape 2 : Déploiement d'un réseau virtuel et d'un conteneur à l'aide de Docker compose

Dans votre instance AWS,

- Récupérez l'archive dont l'URL est fournie sur Moodle sous l'activité "Exemple Compose". Sauvegardez-la sous le nom ***sshd-uqac.tar***. Son checksum sha-256 est :
`fd19317da832807dbb48bd419ce61438fbfe07bd27b272113e4d002c54c1a975`
- Décompressez l'archive en utilisant la commande : ***tar xvf sshd-uqac.tar*** et allez dans le répertoire ***sshd-uqac***. Vous devriez y trouver un fichier ***docker-compose.yml***
- Déployez l'environnement tel que défini dans le fichier yaml précédent en tapant simplement : ***docker-compose up -d***
- Vérifiez avec les commandes ***docker ps*** et ***docker network ls*** les items créés lors de l'exécution de la commande ***docker-compose***
- Analysez le fichier ***docker-compose.yml*** fourni et récupérez l'adresse IP du conteneur ***alpine-sshd***
- À partir de votre instance AWS, essayez de vous connecter en ssh au conteneur ***alpine-sshd*** déployé. Le compte utilisateur est ***labuser***. Le mot de passe est identique.
- Utilisez les connaissances acquises dans la partie précédente afin de trouver une autre façon d'interagir avec le conteneur déployé
- Dans le répertoire ***sshd-uqac***, tapez la commande : ***docker-compose down***
Cela permettra automatiquement d'arrêter et de supprimer les ressources définies dans le fichier yaml

Vous devriez maintenant avoir toutes les bases vous permettant d'utiliser Docker et ses composants dans le contexte des Labs Forensiques.